STM32 GPIO Code Walkthrough

Ankit Srivastava

EE2028

Unless stated otherwise, for the second part of the module,

‘STM32 Chip’ = STM32L4S5VIT6

‘Board’ = B-L4S5I-IOT01A

Acknowledgement: Slides adapted from Dr Rajesh Panicker

OVERVIEW

- GPIO Code Walkthrough

Conclusion

- GPIO Code Walkthrough
  - Configure GPIOs
  - Read/Write GPIOs

UNDERSTANDING GPIO HAL LIBRARY VIA PROJECT

```c
#include "main.h"
static void MX_GPIO_Init(void);

int main(void)
{

  HAL_Init();
  MX_GPIO_Init();

  while (1)
  {
      HAL_GPIO_TogglePin(GPIOB, GPIO_PIN_14);
      HAL_Delay(100);
  }
}

static void MX_GPIO_Init(void)
{
  GPIO_InitTypeDef GPIO_InitStruct = {0};

  /* GPIO Ports Clock Enable */
  __HAL_RCC_GPIOB_CLK_ENABLE();

  /*Configure GPIO pin Output Level */
  HAL_GPIO_WritePin(GPIOB, LED2_Pin, GPIO_PIN_RESET);

  /*Configure GPIO pin LED2_Pin */
  GPIO_InitStruct.Pin = LED2_Pin;
  GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
  GPIO_InitStruct.Pull = GPIO_NOPULL;
  GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
  HAL_GPIO_Init(GPIOB, &GPIO_InitStruct);
}
```

```c
void HAL_GPIO_WritePin(GPIO_TypeDef* GPIOx, uint16_t GPIO_Pin, GPIO_PinState PinState)
{
  /* Check the parameters */
  assert_param(IS_GPIO_PIN(GPIO_Pin));
  assert_param(IS_GPIO_PIN_ACTION(PinState));

  if(PinState != GPIO_PIN_RESET)
  {
    GPIOx->BSRR = (uint32_t)GPIO_Pin;
  }
  else
  {
    GPIOx->BRR = (uint32_t)GPIO_Pin;
  }
}
```

```c
#define GPIO_PIN_14                ((uint16_t)0x4000)  /* Pin 14 selected    */

typedef struct
{
  __IO uint32_t MODER;    /*!< GPIO port mode register,               Address offset: 0x00      */
  __IO uint32_t OTYPER;   /*!< GPIO port output type register,        Address offset: 0x04      */
  __IO uint32_t OSPEEDR;  /*!< GPIO port output speed register,       Address offset: 0x08      */
  __IO uint32_t PUPDR;    /*!< GPIO port pull-up/pull-down register,  Address offset: 0x0C      */
  __IO uint32_t IDR;      /*!< GPIO port input data register,         Address offset: 0x10      */
  __IO uint32_t ODR;      /*!< GPIO port output data register,        Address offset: 0x14      */
  __IO uint32_t BSRR;     /*!< GPIO port bit set/reset  register,     Address offset: 0x18      */
  __IO uint32_t LCKR;     /*!< GPIO port configuration lock register, Address offset: 0x1C      */
  __IO uint32_t AFR[2];   /*!< GPIO alternate function registers,     Address offset: 0x20-0x24 */
  __IO uint32_t BRR;      /*!< GPIO Bit Reset register,               Address offset: 0x28      */
  __IO uint32_t ASCR;     /*!< GPIO analog switch control register,   Address offset: 0x2C      */
} GPIO_TypeDef;
```

```c
#define GPIOB               ((GPIO_TypeDef *) GPIOB_BASE)
#define GPIOB_BASE            (AHB2PERIPH_BASE + 0x0400UL)
#define AHB2PERIPH_BASE       (PERIPH_BASE + 0x08000000UL)
#define PERIPH_BASE           (0x40000000UL) /*!< Peripheral base address */
```

```c
typedef enum
{
  GPIO_PIN_RESET = 0U,
  GPIO_PIN_SET
}GPIO_PinState;
```

```c
void HAL_GPIO_TogglePin(GPIO_TypeDef* GPIOx, uint16_t GPIO_Pin)
{
  /* Check the parameters */
  assert_param(IS_GPIO_PIN(GPIO_Pin));

  if ((GPIOx->ODR & GPIO_Pin) != 0x00u)
  {
    GPIOx->BRR = (uint32_t)GPIO_Pin;
  }
  else
  {
    GPIOx->BSRR = (uint32_t)GPIO_Pin;
  }
}
```

```c
#define LED2_Pin GPIO_PIN_14
typedef struct
{
  uint32_t Pin;        /*!< Specifies the GPIO pins to be configured.*/
  uint32_t Mode;       /*!< Specifies the operating mode for the selected pins.*/
  uint32_t Pull;       /*!< Specifies the Pull-up or Pull-Down activation for the selected pins.*/
  uint32_t Speed;      /*!< Specifies the speed for the selected pins.*/
  uint32_t Alternate;  /*!< Peripheral to be connected to the selected pins*/
} GPIO_InitTypeDef;
```